PSTAT194CS_SNA_Final_Project

Dataset

Loading and Preparing Data

Understanding our Network

```
## Number of Nodes: 875713
```

```
## Number of Edges: 5105039
```

```
## [1] TRUE
```

```
## [1] FALSE
```

```
## IGRAPH 1be632a DN-- 875713 5105039 --
## + attr: name (v/c)
```

Creating Subgraphs of 200-nodes

Code Implemenation

Checking Subgraphs

Random Walk New Implementation

Checking our Subgraphs

```
## IGRAPH 02328f1 DN-- 200 957 --
## + attr: name (v/c)
## + edges from 02328f1 (vertex names):
##  [1] 597621->206688 597621->857417 597621->137136 597621->729403 168595->41909
##  [6] 425770->877707 206688->597621 206688->729403 206688->633614 206688->653868
## [11] 206688->445787 206688->587332 206688->339011 206688->342737 109173->597621
## [16] 109173->292484 109173->355850 109173->413427 109173->857417 109173->866840
## [21] 292484->597621 292484->109173 292484->355850 292484->413427 292484->857417
## [26] 355850->597621 355850->109173 355850->292484 355850->413427 355850->857417
## [31] 355850->614886 355850->649922 355850->660778 355850->233861 355850->83049
## [36] 355850->453644 413427->597621 413427->109173 413427->292484 413427->355850
## + ... omitted several edges
```

```
## IGRAPH 635b286 DN-- 200 970 --
## + attr: name (v/c)
## + edges from 635b286 (vertex names):
##  [1] 185067->76969  274149->185067 274149->425769 274149->504769 274149->572100
##  [6] 274149->406923 274149->485986 274149->129775 274149->314621 274149->839398
## [11] 274149->893642 274149->308700 274149->212718 425769->185067 425769->274149
## [16] 425769->504769 504769->185067 776322->185067 776322->274149 776322->425769
## [21] 776322->504769 776322->572100 776322->212718 568997->495084 1796  ->430387
```

```
## [26] 430387->495084 430387->1796   430387->293063 430387->787593 430387->735531
## [31] 419525->506762 419525->741326 419525->151764 506762->495084 506762->419525
## [36] 506762->741326 506762->50817  741326->495084 741326->419525 741326->506762
## + ... omitted several edges
```

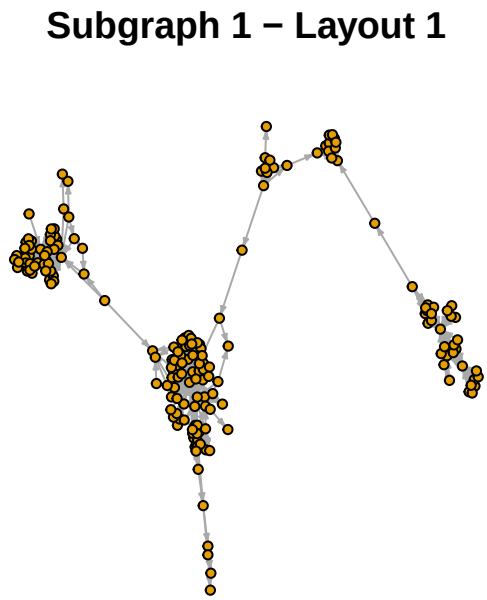
```
## IGRAPH 1c3a8fd DN-- 200 842 --
## + attr: name (v/c)
## + edges from 1c3a8fd (vertex names):
##  [1] 163075->804489 163075->394956 163075->858447 163075->638031 163075->156398
##  [6] 163075->639446 588614->163075 804489->790474 393008->163075 393008->555037
## [11] 393008->688710 555037->163075 555037->393008 555037->688710 555037->554409
## [16] 688710->163075 688710->393008 688710->555037 688710->229271 790474->804489
## [21] 790474->59788  896105->246439 896105->304260 99074 ->804489 99074 ->790474
## [26] 99074 ->59788  11350 ->721816 11350 ->523821 11350 ->827541 11350 ->698356
## [31] 523821->721816 523821->11350  523821->773477 523821->827541 773477->721816
## [36] 773477->11350  773477->827541 827541->721816 827541->773477 827541->246656
## + ... omitted several edges
```

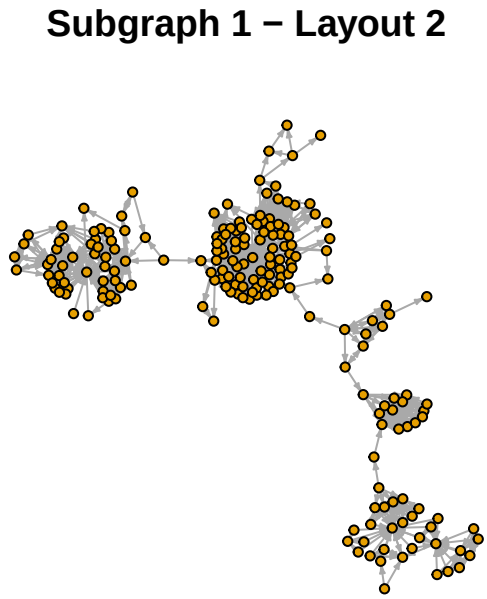
```
## IGRAPH bc73db8 DN-- 200 1608 --
## + attr: name (v/c)
## + edges from bc73db8 (vertex names):
##  [1] 223236->421091 223236->425667 223236->822292 78244 ->599130 147159->599130
##  [6] 147159->102796 599130->213338 599130->78244  599130->147159 599130->541901
## [11] 366369->223236 366369->369637 366369->463481 366369->495374 369637->223236
## [16] 369637->366369 369637->463481 369637->495374 369637->55159  463481->223236
## [21] 463481->366369 463481->369637 463481->495374 463481->599129 495374->223236
## [26] 495374->366369 495374->369637 495374->463481 787300->223236 92139 ->75783
## [31] 92139 ->85406  92139 ->435217 92139 ->441612 92139 ->506789 92139 ->545842
## [36] 92139 ->702627 92139 ->737494 92139 ->832513 92139 ->880108 814716->223236
## + ... omitted several edges
```

Checking on Sampling Bias

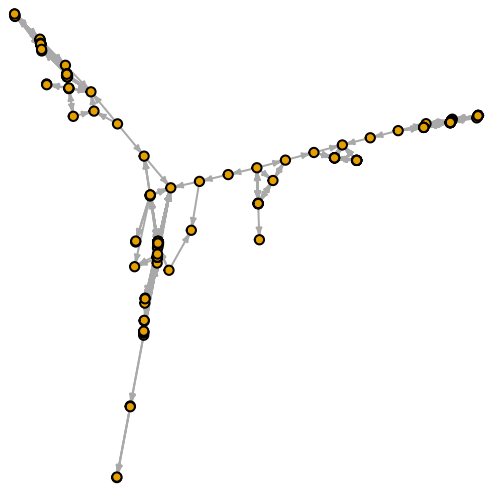
```
##     Subgraph Nodes Edges Mean_Degree Median_Degree   KS_p_value
## 1 Subgraph 1   200   957        9.57             7 3.368043e-05
## 2 Subgraph 2   200   970        9.70             5 6.265071e-03
## 3 Subgraph 3   200   842        8.42             6 4.668214e-06
## 4 Subgraph 4   200  1608       16.08            12 9.144355e-17
```

Visualization

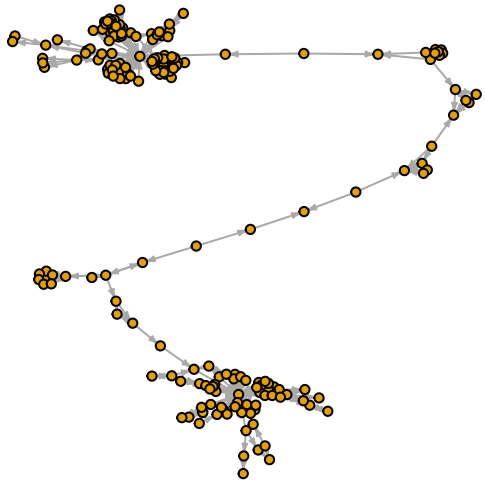




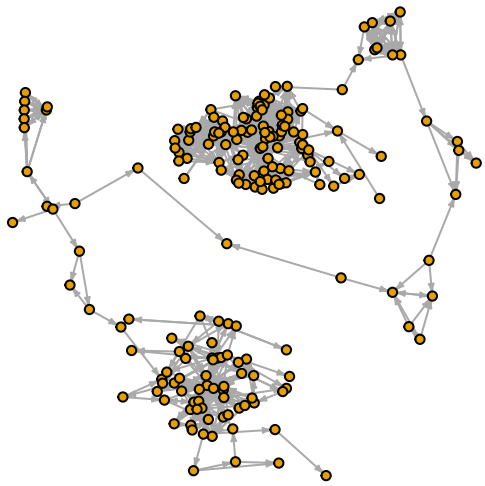
Subgraph 1 − Layout 3



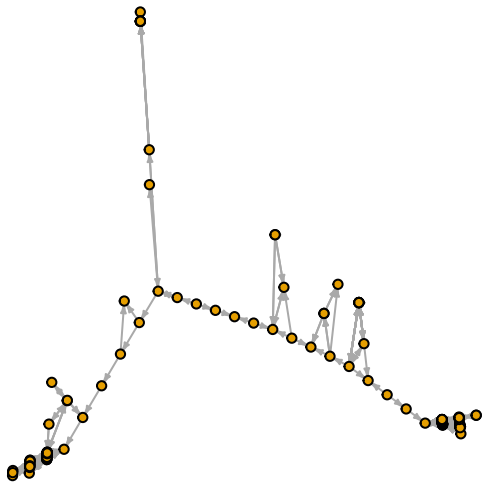
Subgraph 2 − Layout 1



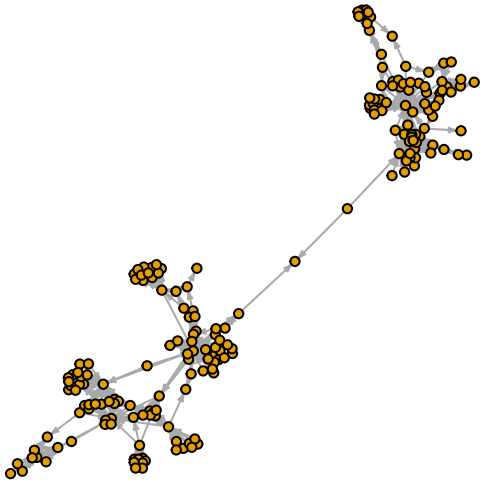
Subgraph 2 − Layout 2



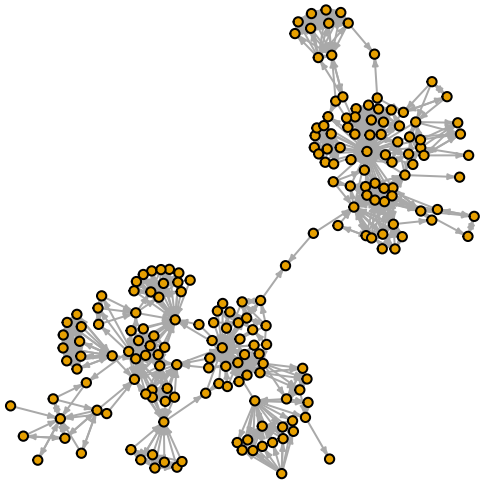
Subgraph 2 − Layout 3



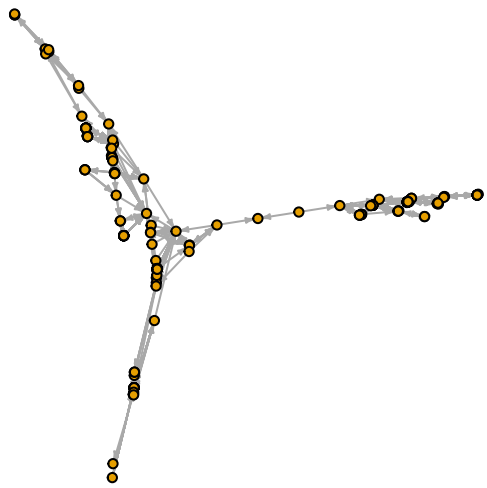
Subgraph 3 − Layout 1



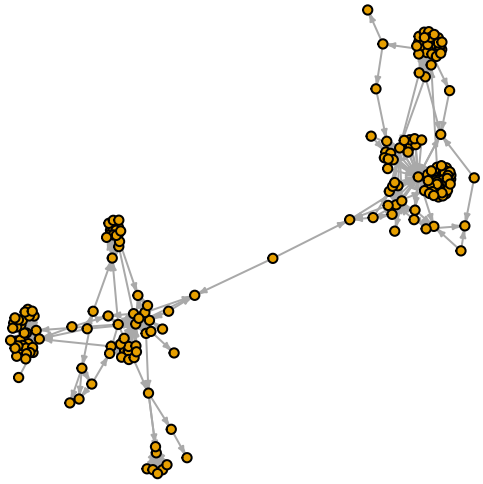
Subgraph 3 − Layout 2



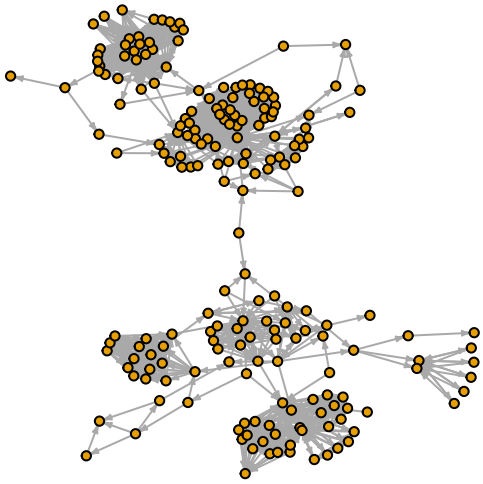
Subgraph 3 − Layout 3

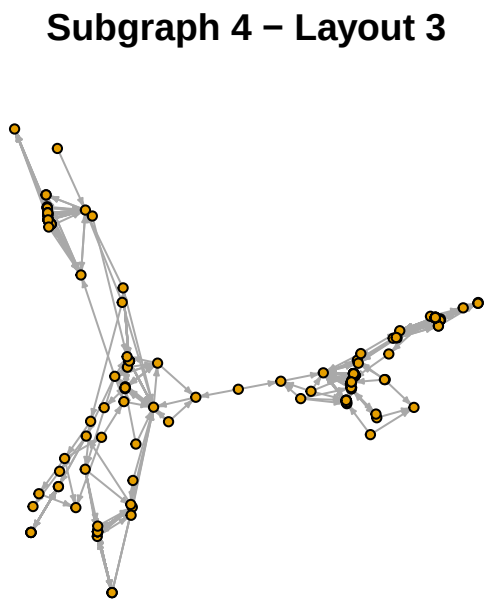


Subgraph 4 − Layout 1



Subgraph 4 − Layout 2





Metrics Evaluated for ALL 4 SUBGRAPHS (probably don't do this)

5) Community Detection

Because the community detection algorithms want an undirected graph and our current graph is directed, we need to convert it into an undirected graph first. Then, we can run all of the community detection algorithms on each subgraph and store it all inside one dataframe. We will also store the results inside a comunity objects data structure so that we can use the results later.

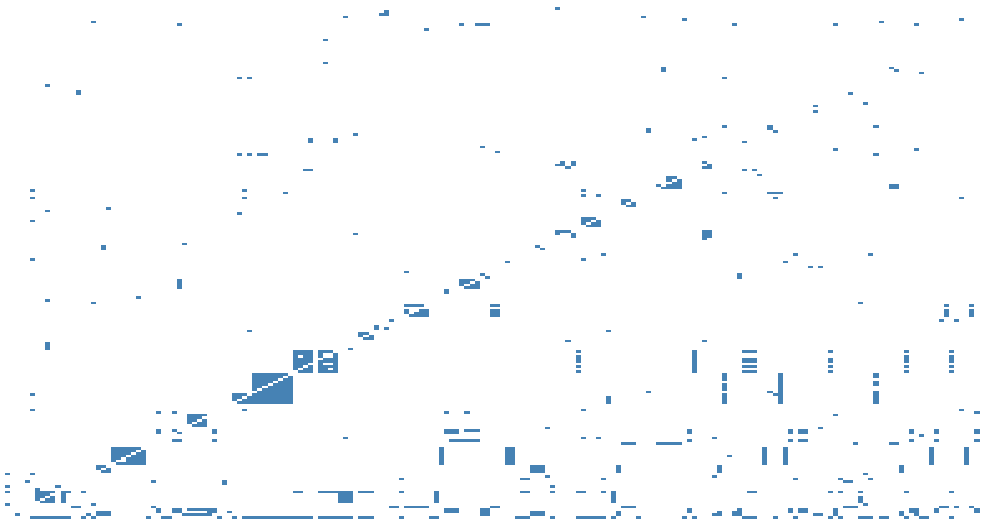
```
##    Subgraph   Algorithm N_communities Modularity_Q Max_Comm_Size
## 1         1 Fast Greedy             8       0.7267            56
## 2         1     Louvain             8       0.7462            51
## 3         1    Walktrap            10       0.7346            66
## 4         2 Fast Greedy             8       0.7086            44
## 5         2     Louvain             9       0.7090            47
## 6         2    Walktrap            20       0.6767            39
## 7         3 Fast Greedy             8       0.7685            45
## 8         3     Louvain            10       0.7886            35
## 9         3    Walktrap            10       0.7818            38
## 10        4 Fast Greedy             7       0.7436            40
## 11        4     Louvain             8       0.7396            38
## 12        4    Walktrap            12       0.7357            38
```

Looking at the results, we see that Walktrap generally detects more communities than the other 2 algorithms.

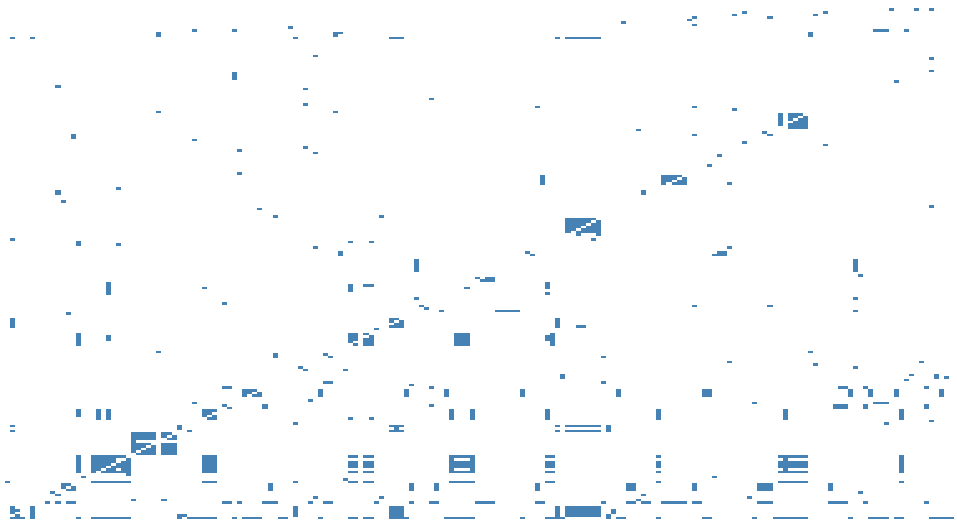
All of the algorithms have high modularity greater than 0.7, suggesting that they found clear communities.

6) Adjacency Matrix

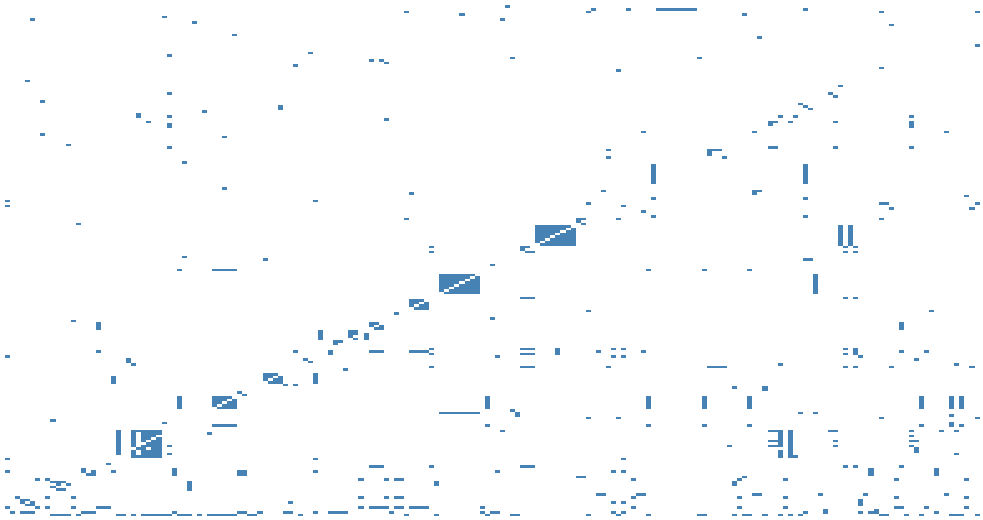
Subgraph 1 − Original Node Order



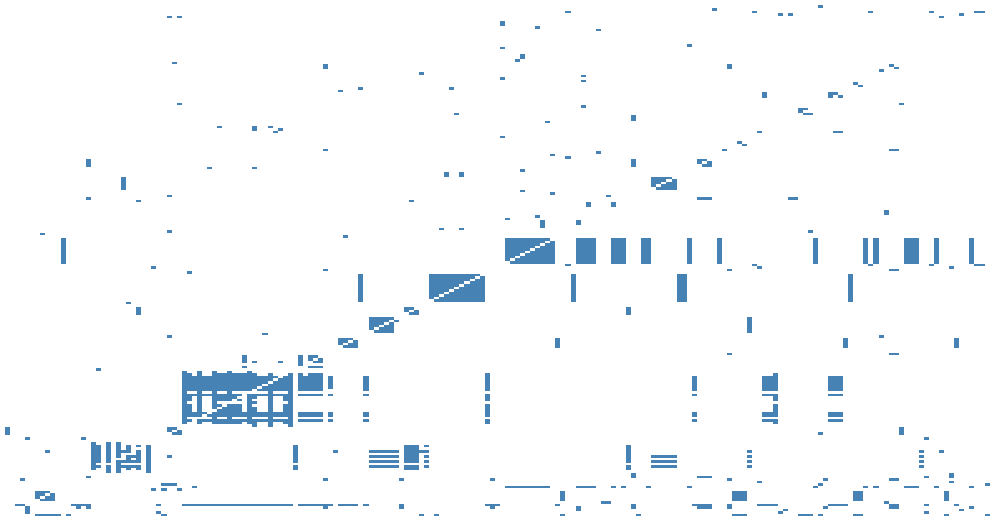
Subgraph 2 − Original Node Order



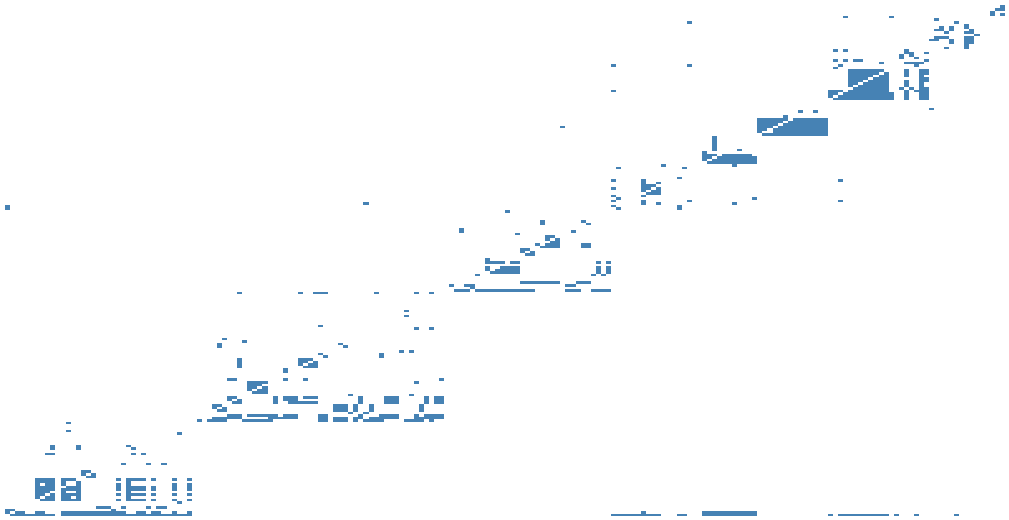
Subgraph 3 − Original Node Order



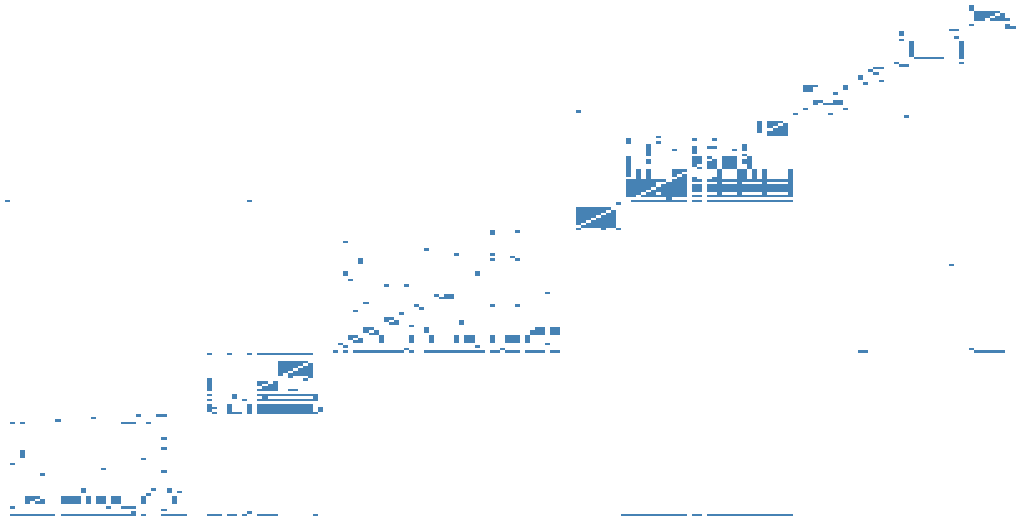
Subgraph 4 − Original Node Order



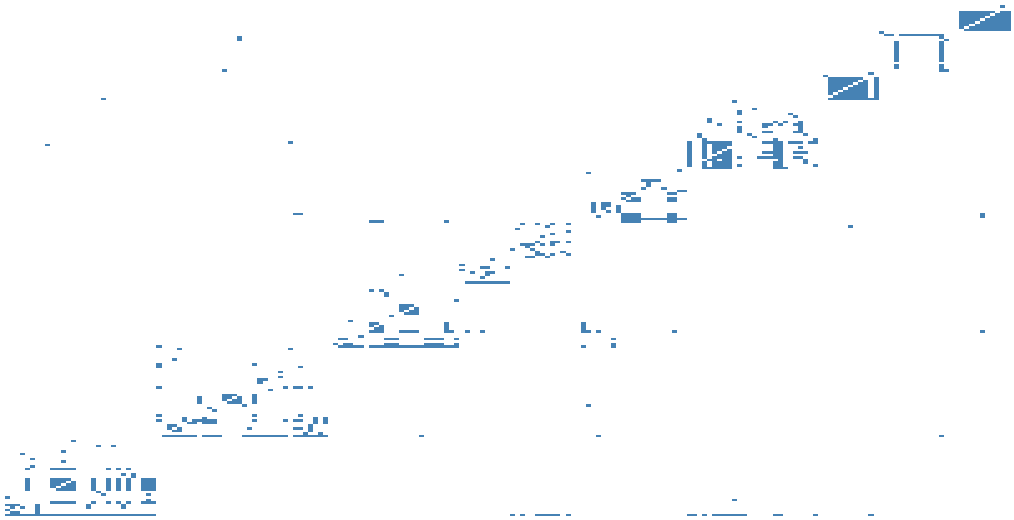
Subgraph 1 − Reordered by Community

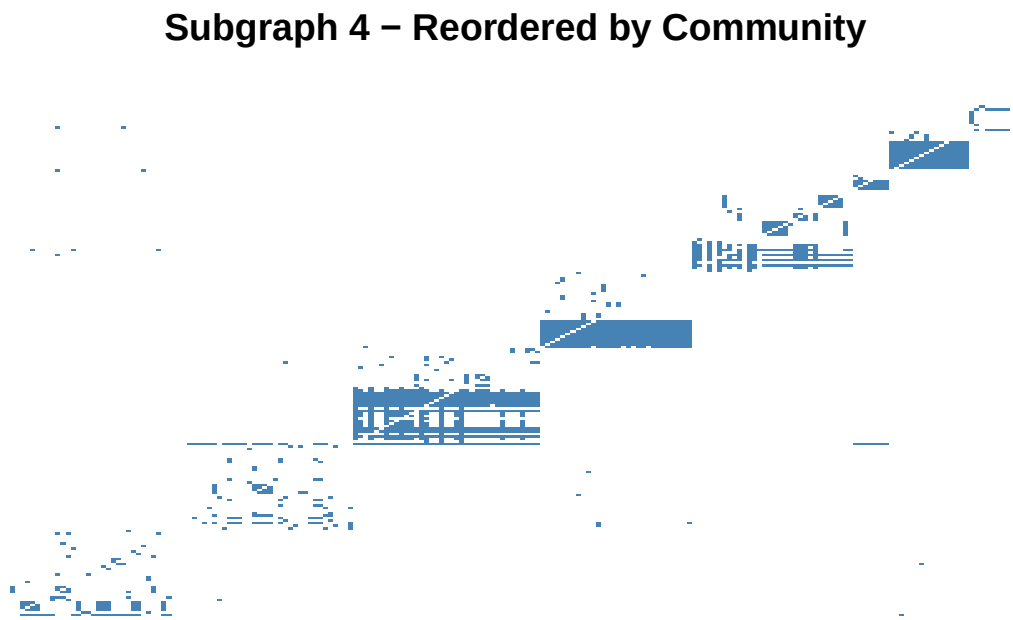


Subgraph 2 − Reordered by Community



Subgraph 3 − Reordered by Community

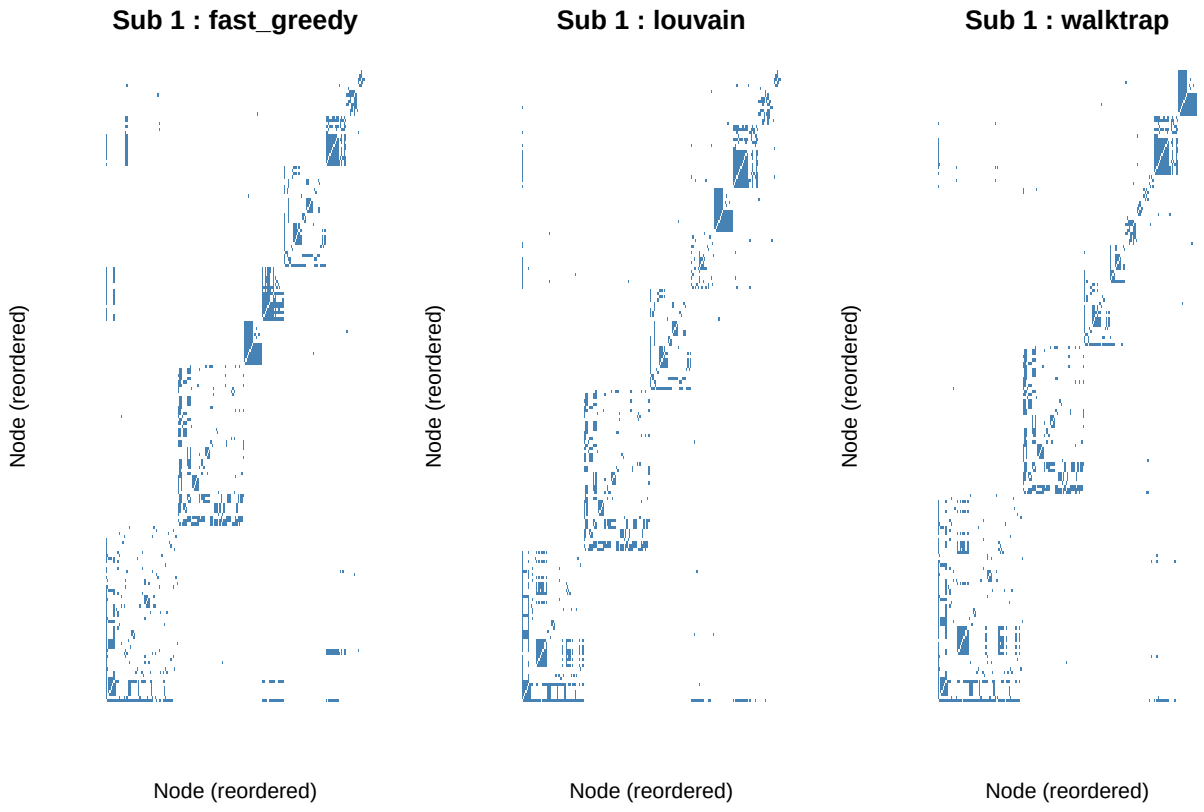


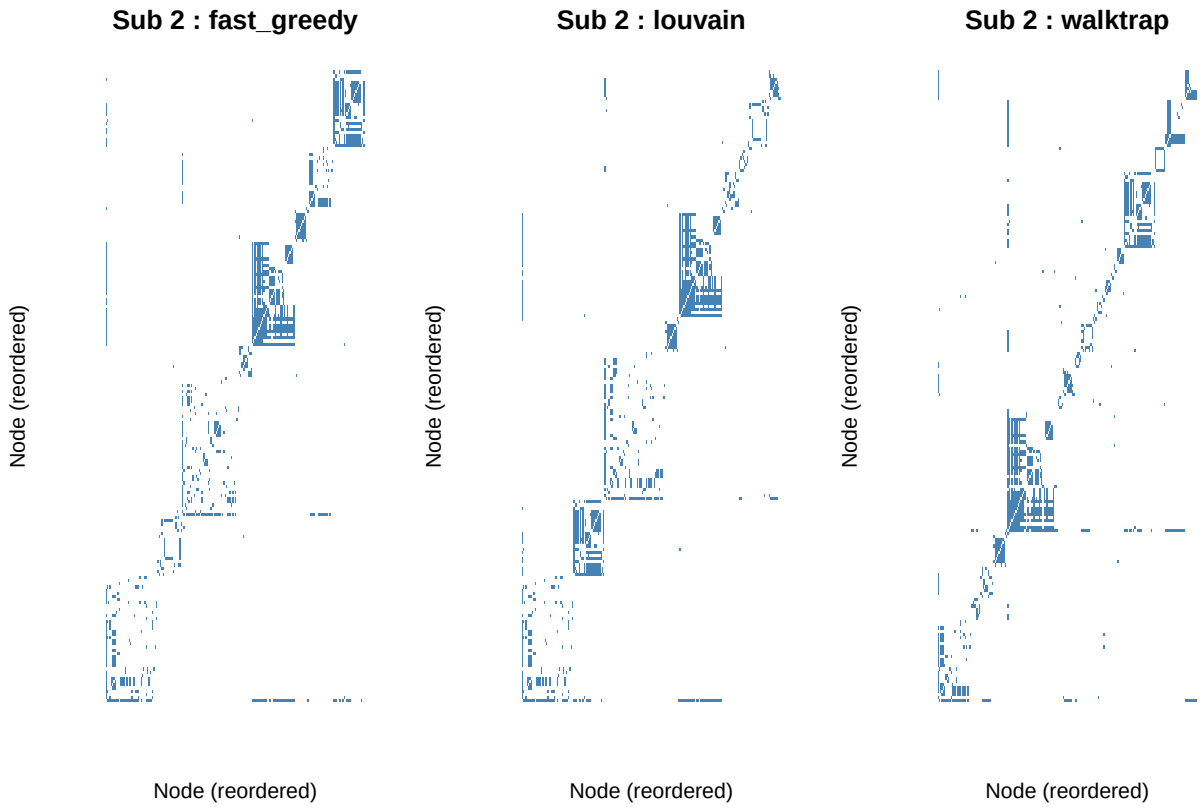


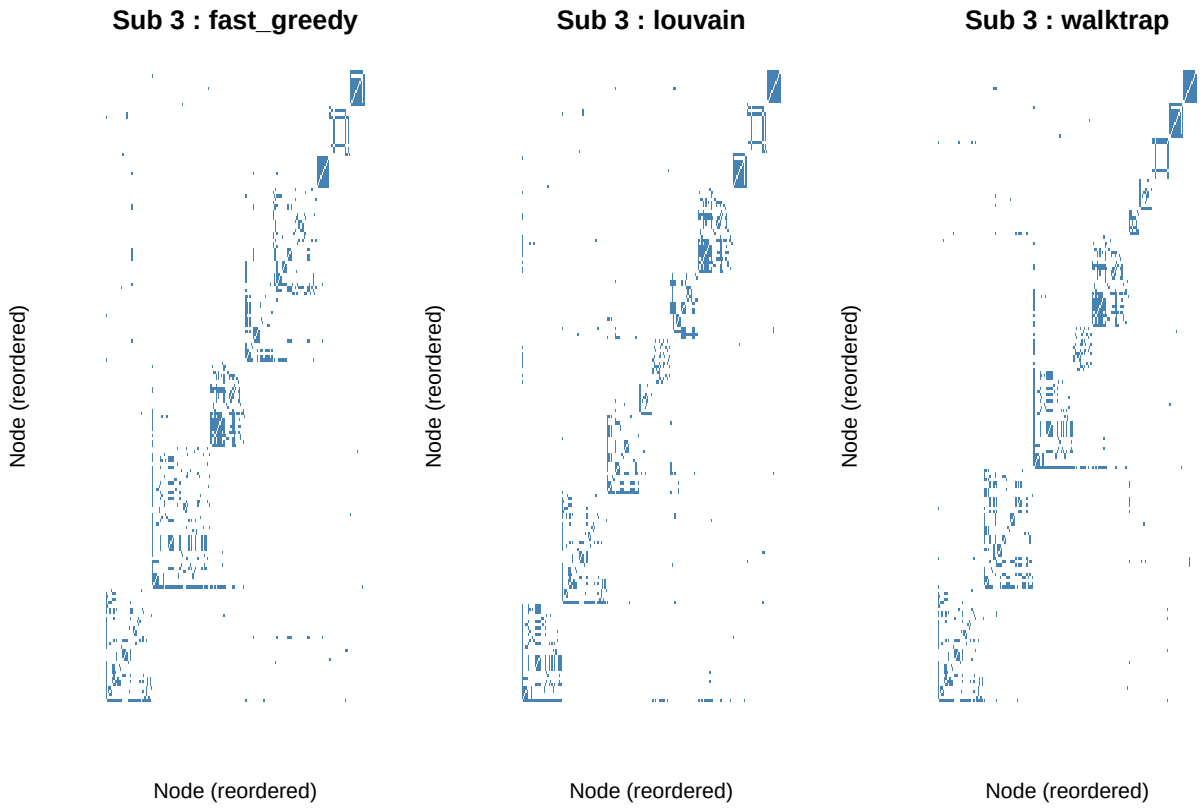
The adjacency matrix of each 200-node subgraph was examined to identify structural patterns in the Google web graph. Each entry in the matrix represents the presence of a directed hyperlink between two pages. In the original node ordering, edges appear scattered with no discernible pattern, reflecting the arbitrary order in which nodes were sampled. To reveal underlying structure, nodes were reordered by Louvain community membership. After reordering, clear diagonal block patterns emerged, particularly in Subgraph 4 which contained 1,608 edges. Each block along the diagonal represents a group of web pages that densely hyperlink to one another, likely corresponding to topically related pages. The sparsity off the diagonal indicates that pages from different communities rarely link directly to each other. This visual confirmation aligns with the community detection results, supporting the presence of meaningful cluster structure within the Google web graph.

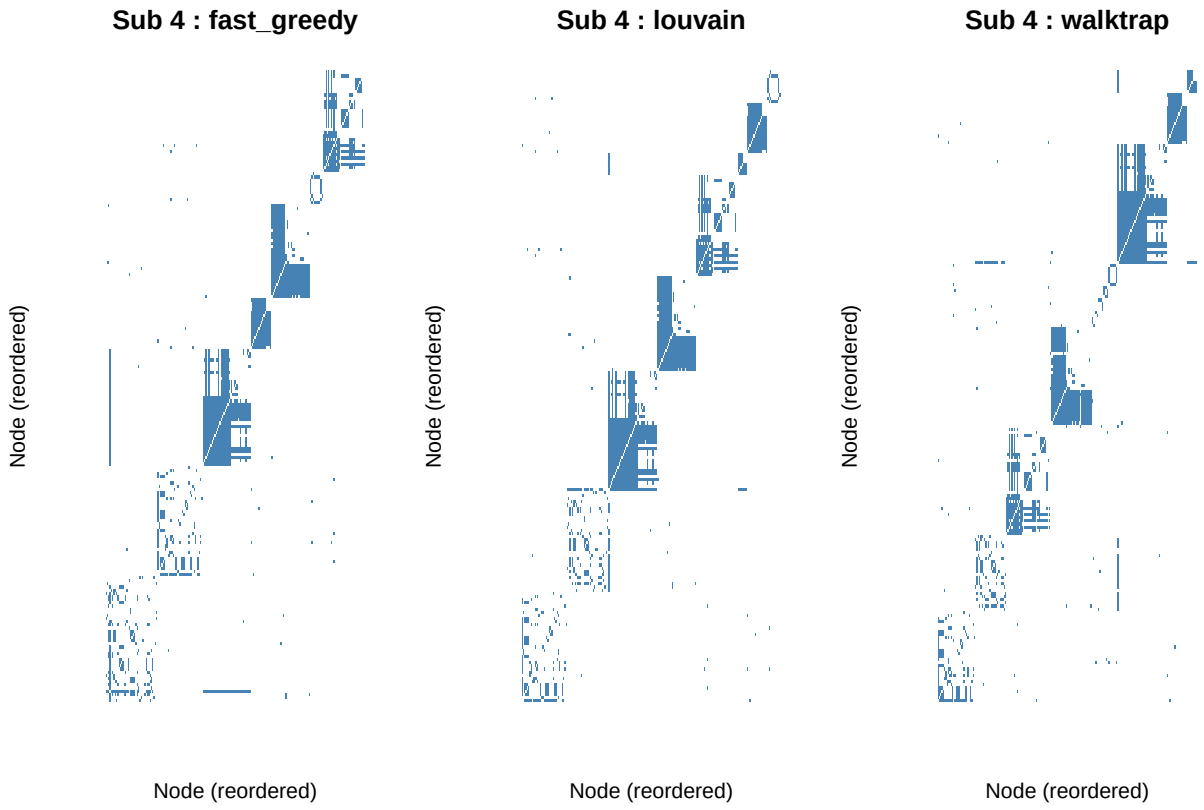
7) Reordering Nodes for Community Detection

As we see above, all of the nodes are in one connected component. As such, this step is essentially the same as above. We will reorder the adjacency matrices for all of the remaining subgraphs as well.


Sub 1 : fast_greedy
Sub 1 : louvain
Sub 1 : walktrap
Node (reordered)
Node (reordered)
Node (reordered)
Node (reordered)
Node (reordered)
Node (reordered)


Sub 2 : fast_greedy
Sub 2 : louvain
Sub 2 : walktrap
Node (reordered)
Node (reordered)
Node (reordered)
Node (reordered)
Node (reordered)
Node (reordered)


Sub 3 : fast_greedy
Sub 3 : louvain
Sub 3 : walktrap
Node (reordered)
Node (reordered)
Node (reordered)
Node (reordered)
Node (reordered)
Node (reordered)



After reordering by community for each algorithm, we can see that the nodes form multiple boxes. This represents communities where nodes are highly interconnected. The adjacency matrix also happens to be mirrored across the y=x line, and that's because we used the undirected graph. We chose to use the undirected graph instead of the directed graph because that is the graph the algorithms used to detect the communities. If you also compare the adjencency matrix to the algorithm results, you see that the boxes correspond to how many communities each algorithm detected.

There are two main shapes we see in the adjacency matrix. First is this sparse L shape. This represents a central node that connects to many other nodes. In the context of Google, this could be a site like Wikipedia that has links to many different communities. Next is a solid box. This represents a community where all the nodes are closely interconnected. This could be a set of subpages, where all pages link to each other.

Appendix

```
knitr::opts_chunk$set(
  echo = FALSE,   # hides code
  message = FALSE,
  warning = FALSE
)
url = "https://snap.stanford.edu/data/web-Google.txt.gz"
download.file(url, destfile = "web-Google.txt.gz", mode = "wb")
library(igraph)

edges = read.table(
  "web-Google.txt.gz",
  comment.char = "#",
)

graph = graph_from_data_frame(edges, directed=TRUE)
cat("Number of Nodes:",vcount(graph),"\n")
cat("Number of Edges:",ecount(graph))
is_directed(graph)
is_weighted(graph)
summary(graph)
set.seed(123)

sub.network = lapply(1:4, function(x){
  induced_subgraph(graph, sample(V(graph),200))
})
sub.network[[1]]
sub.network[[2]]
sub.network[[3]]
sub.network[[4]]
set.seed(123)

subgraphs <- list()

for(i in 1:4){
  start_node <- sample(V(graph), 1)
  walk <- random_walk(graph,
                      start = start_node,
                      steps = 500,
                      mode = "all")
  nodes_sampled <- unique(walk)
  nodes_sampled <- nodes_sampled[1:200]
  subgraphs[[i]] <- induced_subgraph(graph, nodes_sampled)
}
subgraphs[[1]]
subgraphs[[2]]
subgraphs[[3]]
subgraphs[[4]]
deg.full = degree(graph)


bias.results = data.frame(
  Subgraph = character(),
```

```r
  Nodes = numeric(),
  Edges = numeric(),
  Mean_Degree = numeric(),
  Median_Degree = numeric(),
  KS_p_value = numeric(),
  stringsAsFactors = FALSE
)

for (i in 1:4){
  deg.sample = degree(subgraphs[[i]])
  ks = ks.test(deg.sample, deg.full)
  bias.results = rbind(
    bias.results,
    data.frame(
      Subgraph = paste("Subgraph", i),
      Nodes = vcount(subgraphs[[i]]),
      Edges = ecount(subgraphs[[i]]),
      Mean_Degree = mean(deg.sample),
      Median_Degree = median(deg.sample),
      KS_p_value = ks$p.value
      )
    )
}
bias.results
layouts = list(layout_with_fr, layout_with_kk, layout_with_mds)

for(i in 1:4){
  for (j in 1:3){
    plot(
      subgraphs[[i]],
      layout = layouts[[j]],
      vertex.size = 4,
      vertex.label = NA,
      edge.arrow.size = 0.2,
      main= paste("Subgraph", i,"- Layout", j)
    )
  }
}
for (i in 1:4) {
  cat("Metrics for Subgraph", i, ":\n", "NETWORK LEVEL METRICS", "\n")

  cat("Density:", "\n")
  print(edge_density(subgraphs[[i]]))

  # Reciprocity
  cat("Reciprocity:", "\n")
  print(reciprocity(subgraphs[[i]]))

  # Dyad Census
  cat("Pairwise Relationships:", "\n")
  print(dyad_census(subgraphs[[i]]))

  # Transitivity
```

```r
  cat("Transitivity:", "\n")
  print(transitivity(subgraphs[[i]]))

  # Average distance between nodes
  cat("Mean Distance Between Nodes:", "\n")
  print(mean_distance(subgraphs[[i]]))

  # Diameter
  cat("Diameter and the Associated Nodes:", "\n")
  print(diameter(subgraphs[[i]], weights = NA))
  print(farthest_vertices(subgraphs[[i]]))

  # NODE LEVEL METRICS
  cat("NODE LEVEL METRICS", "\n", "Degree:", "\n")
  print(degree(subgraphs[[i]], mode = "in"))

  # Centrality
  cat("Betweenness Centrality:", "\n")
  print(betweenness(subgraphs[[i]]))
  cat(mean(is.nan(betweenness(subgraphs[[i]]))), "of the values are NaN", "\n")

  cat("Closeness Centrality:", "\n")
  clr <- centr_clo(subgraphs[[i]], mode="all", normalized=T)
  print(centr_clo(subgraphs[[i]], mode="all", normalized=T))
  cat(mean(is.nan(closeness(subgraphs[[i]]))), "of the values are NaN", "\n")
  sort(clr$res, decreasing = T, index.return = T) # REMOVE ME Can remove if we don't want to plot

  cat("Local Transitivity:", "\n")
  print(transitivity(subgraphs[[i]], type="local"))

  # Edge Level
  edge_betweenness(subgraphs[[i]])
}
# convert all into undirected graphs
undirected_subgraphs <- vector("list", 4)
for (i in 1:4){
  undirected_subgraphs[[i]] <- as.undirected(subgraphs[[i]], mode="collapse")
}
# list to store all results
all_results <- list()
community_objects <- list()

for (i in 1:4) {
  g <- undirected_subgraphs[[i]]

  community_objects[[i]] <- list(
    fast_greedy = cluster_fast_greedy(g),
    louvain     = cluster_louvain(g),
    walktrap    = cluster_walktrap(g)
  )

  # Grab the objects we just saved to calculate stats
  objs <- community_objects[[i]]
```

```r
  all_results[[i]] <- data.frame(
    Subgraph      = i,
    Algorithm     = c("Fast Greedy", "Louvain", "Walktrap"),
    N_communities = c(length(objs$fast_greedy), length(objs$louvain), length(objs$walktrap)),
    Modularity_Q  = round(c(modularity(objs$fast_greedy), modularity(objs$louvain), modularity(objs$wall
    Max_Comm_Size = c(max(sizes(objs$fast_greedy)), max(sizes(objs$louvain)), max(sizes(objs$walktrap))
  )
}

# combine list into one table
final_comparison <- do.call(rbind, all_results)
final_comparison
for(i in 1:4){
  adj_mat <- as_adjacency_matrix(subgraphs[[i]], sparse = FALSE)

  image(adj_mat,
        main = paste("Subgraph", i, "- Original Node Order"),
        col = c("white", "steelblue"),
        axes = FALSE)
}
set.seed(42)
for(i in 1:4){
  adj_mat <- as_adjacency_matrix(subgraphs[[i]], sparse = FALSE)

  kc_louv <- cluster_louvain(undirected_subgraphs[[i]])
  node_order <- order(membership(kc_louv))
  adj_mat_reordered <- adj_mat[node_order, node_order]

  image(adj_mat_reordered,
        main = paste("Subgraph", i, "- Reordered by Community"),
        col = c("white", "steelblue"),
        axes = FALSE)
}
for (i in 1:4) {
  A <- as_adjacency_matrix(undirected_subgraphs[[i]], sparse = FALSE)

  par(mfrow = c(1, 3))

  # loop through the algorithms
  alg_names <- names(community_objects[[i]])

  for (alg in alg_names) {
    comm_obj <- community_objects[[i]][[alg]]

    ord <- order(membership(comm_obj))

    A_reordered <- A[ord, ord]

    image(A_reordered,
          main = paste("Sub", i, ":", alg),
          xlab = "Node (reordered)",
          ylab = "Node (reordered)",
          col=c("white", "steelblue"),
```

```
          axes = FALSE)
  }

  # reset plots for next one
  par(mfrow = c(1, 1))
}
```